

Spis Treści

1. Wstęp

- 1.1. Cel Projektu
- 1.2. Teza Badawcza
- 1.3. Krótkie Podsumowanie Wyników

2. Metodologia i Architektura Modelu

- 2.1. Charakterystyka Modelu
- 2.2. Zbiór Danych
- 2.3. Funkcja Celu

3. Ekstrakcja Aktywacji

- 3.1. Mechanizm Forward Hooks
- 3.2. Reprezentacja Danych

4. Sondowanie Liniowe (Linear Probing)

- 4.1. Definicja i Trening Sond
- 4.2. Wyniki Detekcji: Fizyka Planszy
- 4.3. Wyniki Detekcji: Taktyka
- 4.4. Zjawisko Zatarcia Liniowego

5. Interwencje Przyczynowe (Causal Interventions)

- 5.1. Definicja Dowodu Przyczynowo-Skutkowego
- 5.2. Wymuszanie Decyzji
- 5.3. Efekt Fali Uderzeniowej
- 5.4. Asymetria Przyczynowa i Samonaprawa
- 5.5. Paradoks Stanu Niemożliwego i Modularność Sieci

6. Podsumowanie i Wnioski

- 6.1. Weryfikacja Tezy Głównej
- 6.2. Odporność Architektury Transformer
- 6.3. Modularność Reprezentacji (Niezależne Obwody)
- 6.4. Znaczenie Projektu dla Dziedziny Interpretowalności AI

1. Wstęp

Zrozumienie, w jaki sposób sztuczne sieci neuronowe przetwarzają informacje i podejmują decyzje, jest obecnie jednym z największych wyzwań w dziedzinie sztucznej inteligencji. Choć modele językowe osiągają niesamowite wyniki, ich wewnętrzne mechanizmy często pozostają nieprzeniknioną "czarną skrzynką". Ten projekt ma na celu rzucenie światła na te ukryte procesy.

1.1. Cel Projektu

Głównym celem pracy jest przeprowadzenie **Mechanistic Interpretability**.

Jako środowisko testowe wybrano grę w Kółko i Krzyżyk (Tic-Tac-Toe). Zbudowano i wytrenowano autoregresyjny model językowy oparty na architekturze Transformer, uczony wyłącznie przewidywania kolejnego prawidłowego ruchu na podstawie płaskiej sekwencji znaków. Projekt skupia się na stworzeniu narzędzi analitycznych, które pozwolą "zajrzeć do wnętrza" tego modelu i zrozumieć, w jaki sposób analizuje on grę bez dostępu do wizualnego obrazu planszy.

1.2. Teza Badawcza

Punktem wyjścia dla naszych badań jest hipoteza o istnieniu **Emergent World Representations**.

Teza projektu brzmi: Autoregresyjny model Transformer, trenowany wyłącznie na losowych, legalnych sekwencjach ruchów (bez implementacji strategii wygrywającej), nie zapamiętuje jedynie statystycznych ciągów cyfr. Aby skutecznie unikać generowania nielegalnych ruchów, model ten samoistnie wykształca w swoich ukrytych warstwach zgeometryzowaną, przestrzenną mapę planszy oraz aktywnie z niej korzysta podczas podejmowania decyzji.

1.3. Krótkie Podsumowanie Wyników

Przeprowadzone eksperymenty dostarczyły twardych dowodów potwierdzających postawioną tezę, ujawniając jednocześnie złożoność wewnętrznej reprezentacji modelu. Najważniejsze wnioski to:

- **Zdekodowanie Mapy Przestrzennej (Linear & Non-linear Probing):** Udowodniono, że model zbudował wewnątrz siebie rozwarstwioną reprezentację planszy. Prosty, fizyczny stan gry (czy pole jest puste, czy zajęte) jest idealnie i liniowo zakodowany w 128-wymiarowej przestrzeni ukrytej (co wykazano za pomocą sond liniowych). Z kolei głębsza, taktyczna świadomość (odróżnienie własnego pionka od pionka przeciwnika) wymagała zastosowania nieliniowych sond (sieci MLP), co dowodzi większej złożoności tego konceptu wewnątrz wag sieci.
- **Asymetria Dowodu Przyczynowo-Skutkowego (Causal Proof):** Przeprowadzono Interwencje Przyczynowe, modyfikując pamięć modelu w czasie rzeczywistym. Wykazano ewidentną asymetrię w jego zachowaniu:
 - Zmiana informacji z "pole puste" na "pole zajęte" zawsze skutecznie blokowała możliwość ruchu w to miejsce, niezależnie od tego, w której warstwie dokonano interwencji.
 - Zmiana informacji z "pole zajęte" na "pole puste" nie przynosiła efektu we wczesnych fazach przetwarzania. Model ignorował sztuczne pozwolenie i nadal odmawiał wykonania ruchu na zajęte pole.
- **Odkrycie Obwodów Zapasowych (Backup Circuits):** Zjawisko ignorowania pozwolenia na ruch stanowi dowód na istnienie potężnych mechanizmów obronnych wewnątrz sieci. Udowodniono, że głowy uwagi (Self-Attention) aktywnie weryfikują zmanipulowaną mapę przestrzenną z fizyczną historią ruchów podaną na wejściu. Widząc, że dane pole zostało już wcześniej zagrane, sieć koryguje "kłamstwo" badacza i twardo blokuje ruch na wszystkich warstwach modelu (L0, L1, L2). Dowodzi to silnej redundancji w architekturze Transformer.
- **Modularność Sieci (Niezależne Obwody):** Odkryto, że sieć dzieli proces myślowy na odseparowane od siebie moduły. Udowodniono, że radar weryfikujący legalność ruchu na mapie przestrzennej działa całkowicie niezależnie od obwodu sędziego, który odpowiada za zakończenie gry.

2. Metodologia i Architektura Modelu

2.1. Charakterystyka Modelu

Do weryfikacji postawionych hipotez wykorzystano minimalistyczny model językowy oparty na architekturze Transformer (decoder-only). Celowo wybrano niewielką sieć, aby proces inżynierii odwrotnej był w pełni kontrolowany i umożliwiał szczegółową analizę aktywacji wszystkich warstw. Model przetwarza grę w Kółko i Krzyżyk jako jednowymiarową sekwencję tokenów, nie posiadając żadnych wbudowanych założeń dotyczących geometrii planszy ani reguł gry poza tymi wynikającymi z danych treningowych.

Parametr	Wartość
Liczba warstw transformera	5
Wymiar ukryty (d_{model})	128

Tak uproszczona architektura pozwala na bezpośrednie śledzenie przepływu informacji pomiędzy warstwami i stanowi dogodny obiekt badań z zakresu mechanistycznej interpretowalności.

2.2. Zbiór Danych

Środowisko treningowe zostało skonstruowane w sposób izolujący mechanikę gry od jakiejkolwiek strategii wygrywającej. Model trenowano wyłącznie na losowo generowanych, lecz w pełni legalnych sekwencjach ruchów.

Najważniejsze właściwości zbioru:

- Brak implementacji strategii wygrywających.
- Brak funkcji nagrody związanej ze zwycięstwem.
- Brak heurystyk oceny pozycji.
- Gry kończą się naturalnie poprzez losowo osiągnięte zwycięstwo lub remis.

Zbiór danych zawierał 10000 wygenerowanych partii, wszystkie były w zbiorze treningowym z uwagi na możliwość wielu poprawnych ruchów dla każdej pozycji i celowe nie przekazywanie modelowi zasad gry.

Dzięki takiemu rygorowi metodologicznemu każda wykryta reprezentacja przestrzenna planszy może być interpretowana jako efekt uboczny rozwiązywania zadania przewidywania legalnych ruchów, a nie rezultat naśladowania strategii obecnych w danych.

2.3. Funkcja Celu (Next-Token Prediction & Legalność)

Proces uczenia opierał się na klasycznym zadaniu przewidywania następnego tokenu (*Next-Token Prediction*). Funkcją kosztu zastosowaną podczas treningu była entropia krzyżowa:

$$L = - \sum y_i \log(\hat{y}_i)$$

gdzie:

- y_i oznacza prawidłowy rozkład docelowy,

- $\hat{y}_i$ oznacza rozkład prawdopodobieństwa generowany przez model.

Do optymalizacji wykorzystano algorytm AdamW z parametrami:

- **Learning Rate:** 0.0002
- **Batch Size:** 10000
- **Liczba epok:** 15

3. Ekstrakcja Aktywacji

3.1. Mechanizm Forward Hooks

Ekstrakcja ukrytych reprezentacji modelu wymaga precyzyjnego podpięcia się pod jego architekturę bez ingerencji w główne wagi. W tym celu wykorzystano mechanizm haczyków (ang. *hooks*), natywnie wbudowany w bibliotekę PyTorch. Do każdej warstwy decyzyjnej modelu (tj. bloków `TransformerEncoderLayer`) przypinana jest funkcja przechwytyjąca za pomocą metody `register_forward_hook`.

Gdy dana warstwa kończy przetwarzać paczkę danych podczas propagacji w przód (*forward pass*), wyjściowy tensor aktywacji jest równolegle przechwytywany. Kluczowe dla stabilności tego procesu jest użycie funkcji `output.detach()`, która odcina pobrany tensor od głównego grafu obliczeniowego. Dzięki temu proces ekstrakcji wewnętrznych "myśli" sieci do zewnętrznej struktury (słownika) przebiega w pełni niezależnie, zapobiegając wyciekom pamięci i pozwalając na jednoczesne przetwarzanie danych z wielu warstw bez jakichkolwiek zakłóceń dla głównych obliczeń modelu.

```
def create_hook(name):
    def hook_fn(modul, input, output):
        przechowywane_dane[name] = output.detach()
    return hook_fn

for i in range(3):
    model.transformer.layers[i].register_forward_hook(create_hook(f'warstwa_{i}'))
```

3.2. Reprezentacja Danych

Surowe dane wejściowe muszą zostać przetransformowane do postaci ułatwiającej modelowi i zewnętrznym sondom dokładną analizę taktyczną. Wewnętrzna reprezentacja przestrzeni gry opiera się na mapowaniu wartości z relatywnej perspektywy gracza wykonującego aktualny ruch:

- **0** – pole puste
- **1** – własny pionek (Mój pionek)
- **2** – pionek przeciwnika (Pionek wroga)

Zastosowanie takiej relatywnej reprezentacji sprawia, że strony konfliktu dynamicznie "obracają się" po każdej turze. Zabieg ten trwale ujednolica kontekst – sieć bazowa oraz sondy klasyfikujące zawsze ewaluują planszę z punktu widzenia strony atakującej w danym kroku. Znacząco ułatwia to dekodowanie mechanizmów uwagi i intencji sieci, która dzięki temu nie musi dodatkowo uczyć się i utrzymywać w pamięci informacji o tym, kto

(krzyżyk czy kółko) jest autorem rozpatrywanego posunięcia. Transformacja stanu na postać relatywną zachodzi z wykorzystaniem operacji tensorowych.

```
def tłumacz_na_relatywna(plansze_surowe):
    real_plane = plansze_surowe % 3
    liczba_jedynek = (real_plane == 1).sum(dim=1, keepdim=True)
    liczba_dwojek = (real_plane == 2).sum(dim=1, keepdim=True)
    ruch_dwojki = liczba_jedynek > liczba_dwojek
    relatywna_plansza = real_plane.clone()
    maska_ruchu = ruch_dwojki.expand_as(real_plane)
    relatywna_plansza[maska_ruchu & (real_plane == 1)] = 2
    relatywna_plansza[maska_ruchu & (real_plane == 2)] = 1
    return relatywna_plansza
```

4. Sondowanie Liniowe (Linear Probing)

4.1. Definicja i Trening Sond (Liniowa vs MLP)

Sondowanie (ang. *probing*) to kluczowa technika z dziedziny mechanistycznej interpretowalności, polegająca na trenowaniu lekkich klasyfikatorów na "zamrożonych" aktywacjach wewnętrznych badanej sieci neuronowej. Celem sondy nie jest partycypacja w procesie decyzyjnym modelu, lecz weryfikacja empiryczna: *czy w danej warstwie znajduje się zdekodowana, użyteczna informacja o badanym zjawisku?*

W ramach eksperymentu zaimplementowano dwa warianty architektoniczne sond:

- **Sonda Liniowa (LinearProbe):** Jednowarstwowa sieć weryfikująca liniową separowalność cech.
- **Sonda Nieliniowa (MLPProbe):** Wielowarstwowy perceptron wykorzystujący nieliniową funkcję aktywacji (ReLU).

```
class LinearProbe(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.layer = nn.Linear(input_size, output_size)

    def forward(self, x):
        return self.layer(x)

class MLPProbe(nn.Module):
    def __init__(self, input_size, output_size):
        super().__init__()
        self.layer = nn.Sequential(
            nn.Linear(input_size, 256),
            nn.ReLU(),
            nn.Linear(256, output_size)
        )
```

```
def forward(self, x):  
    return self.layer(x)
```

4.2. Wyniki Detekcji: Fizyka Planszy (Puste vs Zajęte)

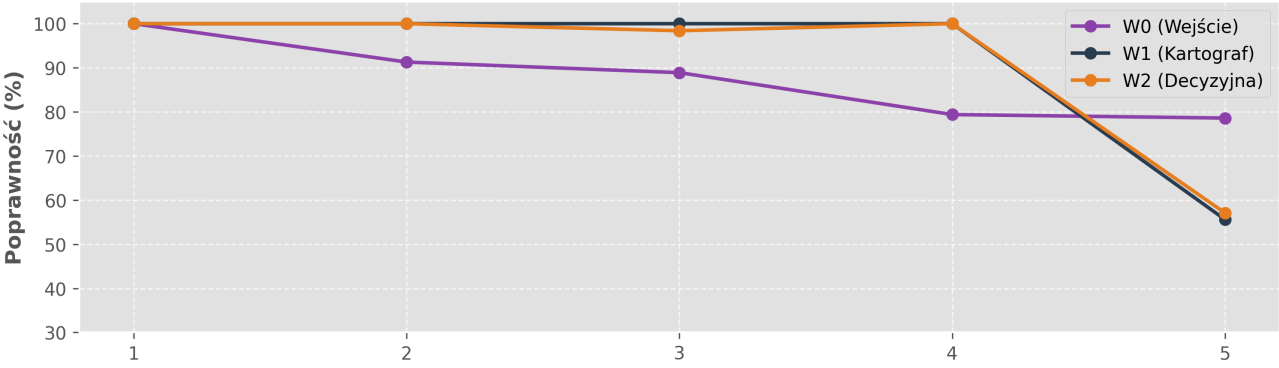
Pierwszym zjawiskiem poddanym analizie była tzw. **"Fizyka Planszy"** – zdolność modelu do odróżnienia pola wolnego od zajętego, niezależnie od tego, do kogo należy pionek. Jest to fundamentalna wiedza wymagana do unikania generowania nielegalnych ruchów.

Zarówno sonda liniowa, jak i MLP osiągnęły w tym zadaniu doskonałą skuteczność (sięgającą niemal 100% w środkowych warstwach modelu). Oznacza to, że pojęcie "pustego pola" jest przez sieć Transformer silnie i jednoznacznie reprezentowane jako geometrycznie oddzielony wektor. Model dosłownie "widzi" fizyczne granice dozwolonego ruchu, a informacja ta jest łatwo dostępna liniowo.

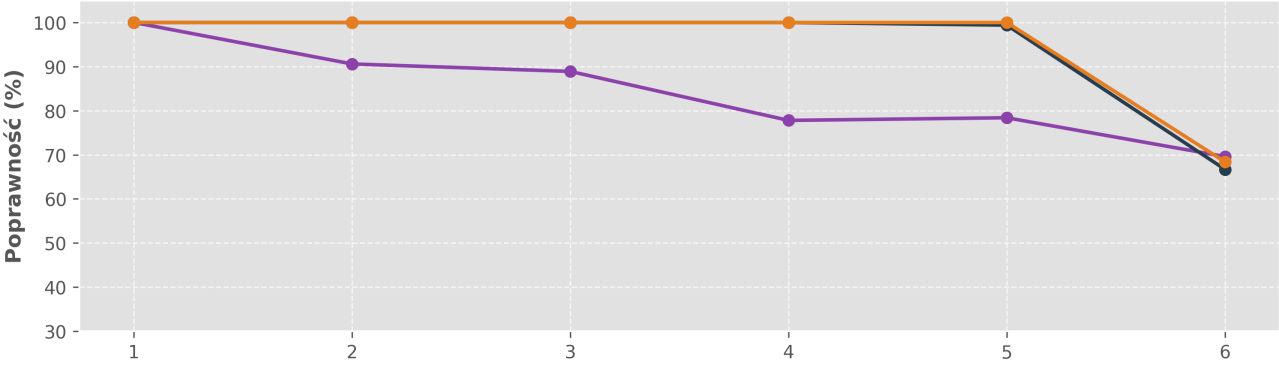
Zauważono jednak interesującą anomalię widoczną na poniższych wykresach. O ile przez większość rozgrywki model doskonale reprezentuje stan całej planszy, o tyle **w samym końcowym etapie gry skuteczność detekcji drastycznie spada**. Dotyczy to zarówno reprezentacji liniowej, jak i nieliniowej (MLP). Zjawisko to wynika z optymalizacji zasobów obliczeniowych i mechaniki kończenia partii. Model nie rozumie koncepcji "zwycięstwa". W decydującym ruchu jego zadaniem jest często jedynie wygenerowanie specjalnego tokenu oznaczającego koniec gry (np. cyfry 9). Ponieważ wytypowanie tego konkretnego znaku nie wymaga już znajomości fizycznego układu wolnych i zajętych pól, mechanizmy sieci po prostu przestają marnować zasoby na utrzymywanie precyzyjnej mapy przestrzennej w pamięci.

Fizyka - SONDA LINIOWA

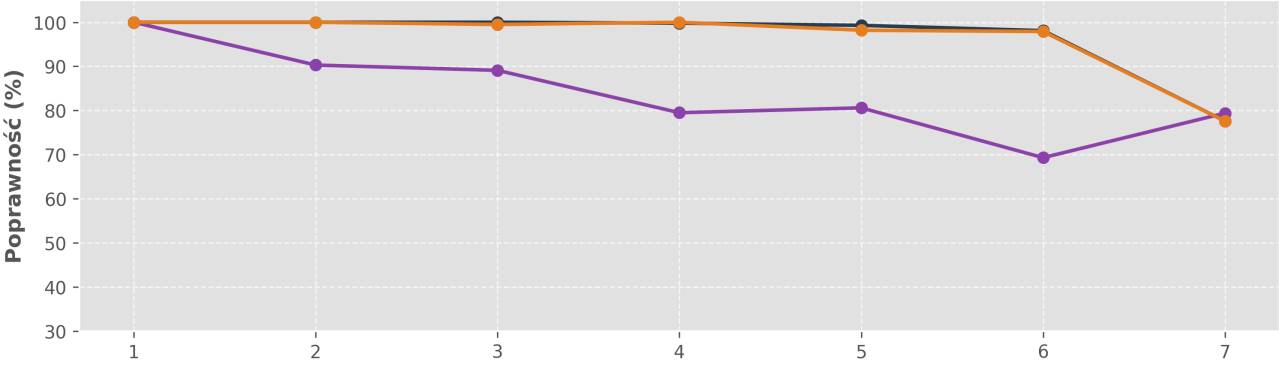
Gry kończące się w 5. ruchu



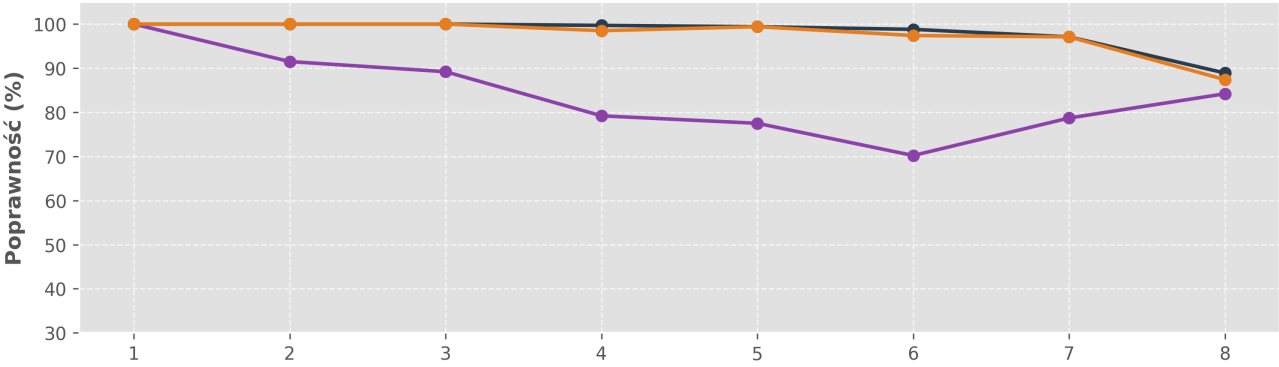
Gry kończące się w 6. ruchu



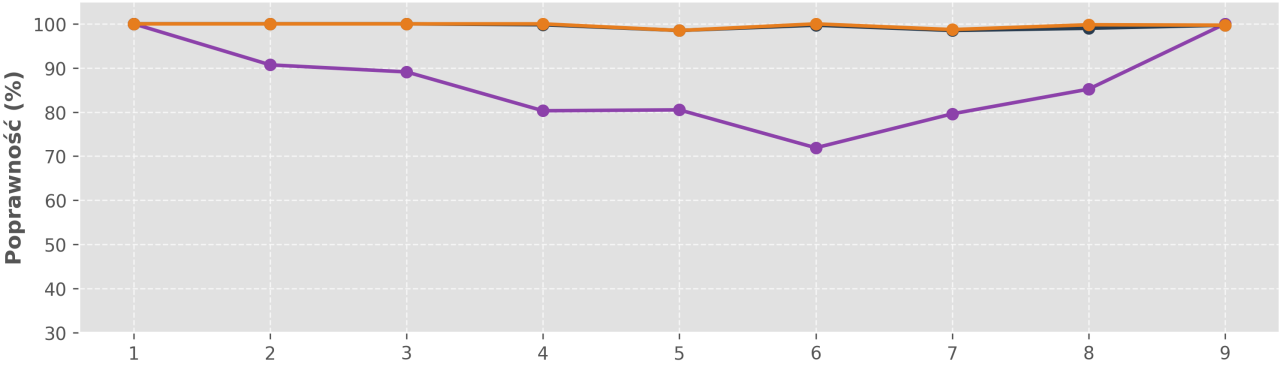
Gry kończące się w 7. ruchu



Gry kończące się w 8. ruchu



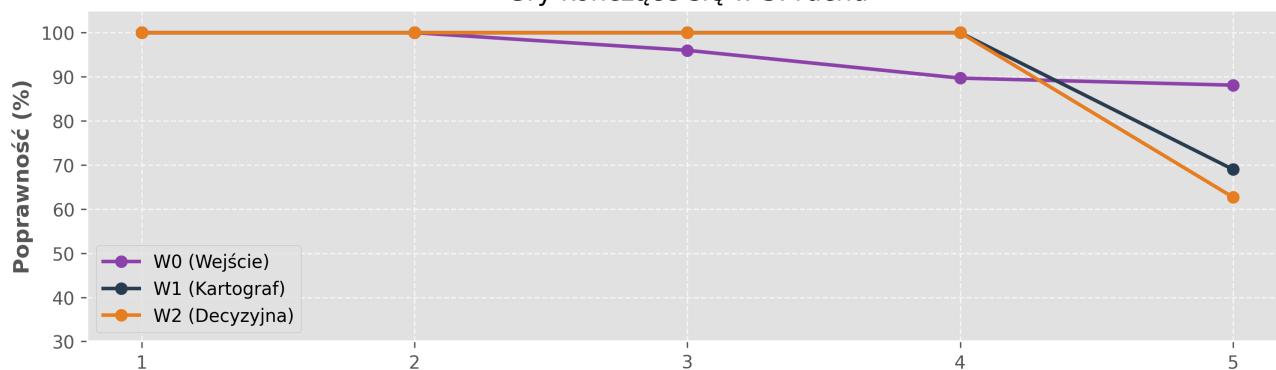
Gry kończące się w 9. ruchu



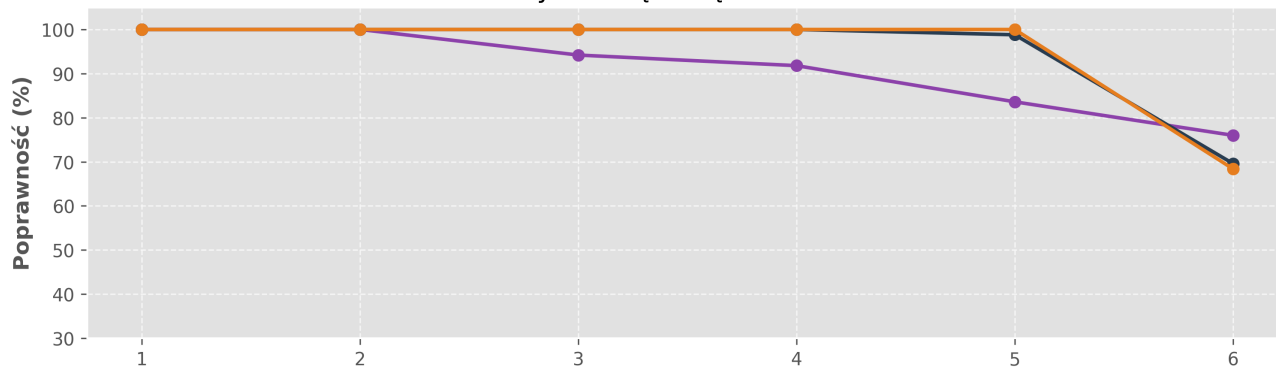
Wykres 1: Skuteczność sondy liniowej w detekcji fizyki planszy. Widoczny wyraźny spadek reprezentacji przestrzennej w momencie, gdy sieć przygotowuje się do wygenerowania tokenu końca gry.

Fizyka - SONDA MLP

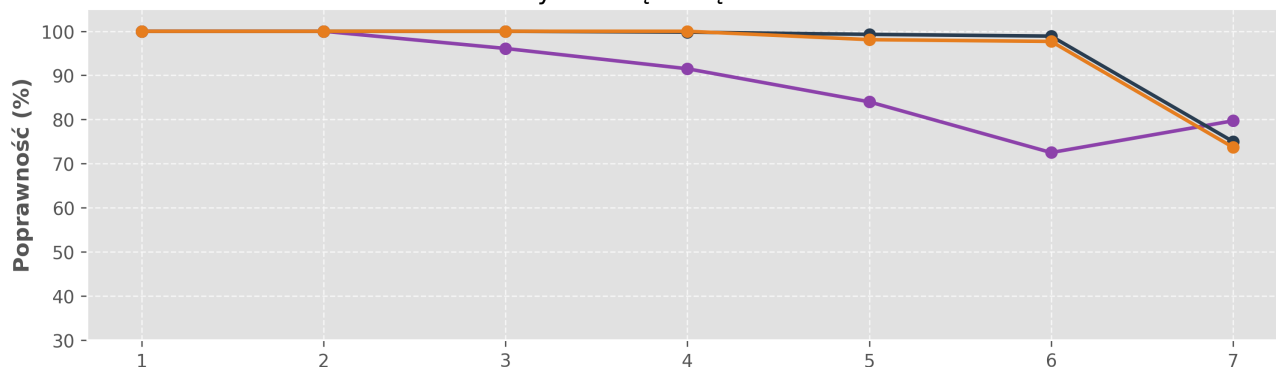
Gry kończące się w 5. ruchu



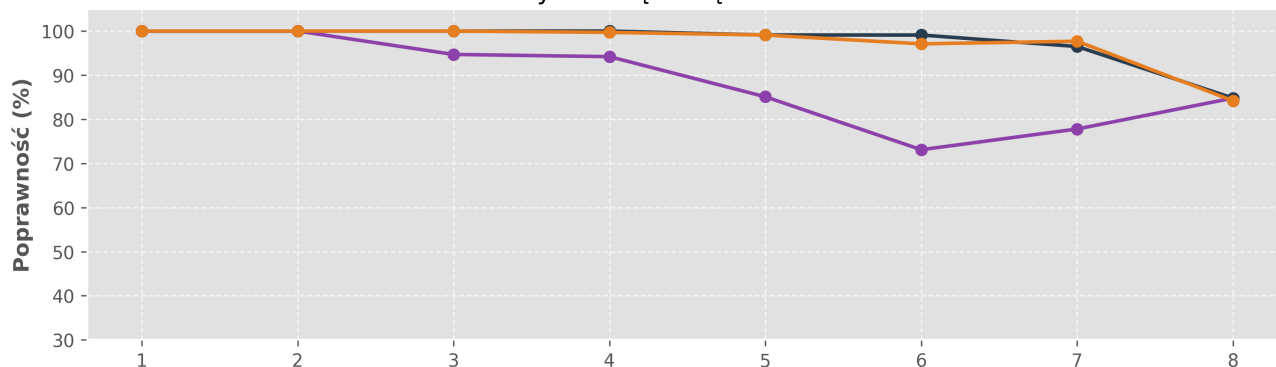
Gry kończące się w 6. ruchu



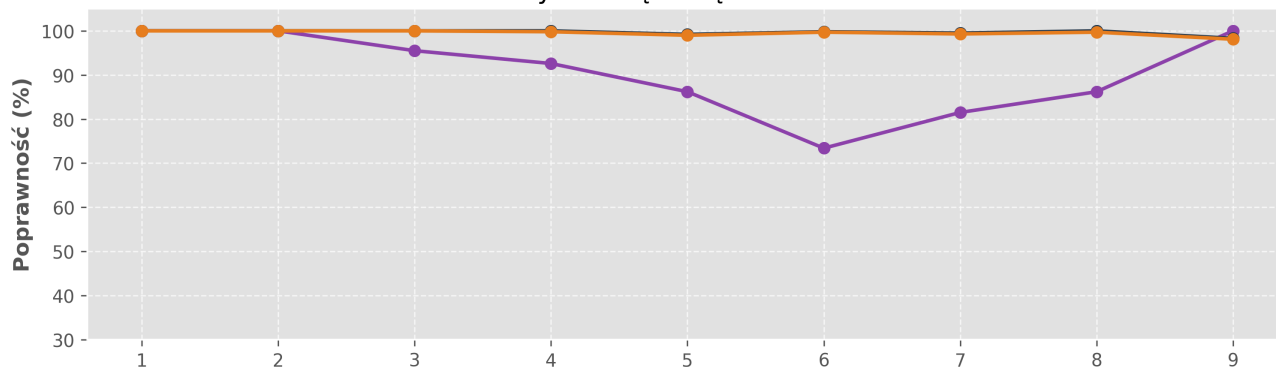
Gry kończące się w 7. ruchu



Gry kończące się w 8. ruchu



Gry kończące się w 9. ruchu



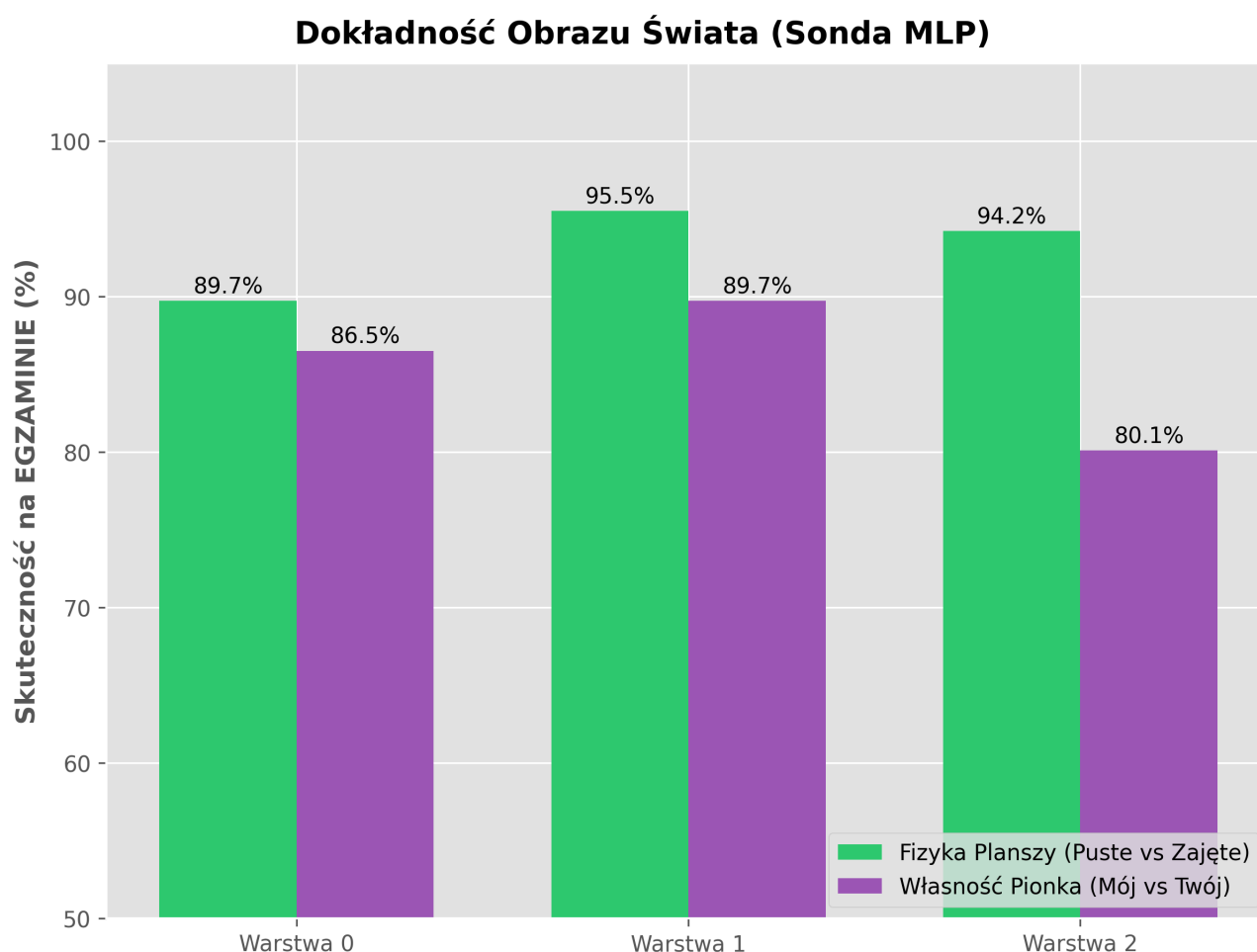
Wykres 2: Skuteczność sondy MLP w detekcji fizyki planszy. Zjawisko wygaszania mapy na końcu partii potwierdza, że sieć aktywnie optymalizuje zasoby i odcina analizę nieistotnych już pustych pól.

4.3. Wyniki Detekcji: Taktyka (Mój Pionek vs Pionek Wroga)

Drugim badanym wymiarem była "**Taktyka**" – zdolność sieci do określenia własności pionka znajdującego się na zajęтым polu (rozróżnienie "Mój" vs "Twój"). To zadanie jest znacznie bardziej złożone, ponieważ wymaga od modelu powiązania historycznego momentu postawienia pionka z identyfikacją gracza.

Wyniki eksperymentu ukazują diametralną różnicę w architekturze wiedzy modelu. **Sonda liniowa radzi sobie z tym zadaniem stosunkowo słabo**, szybko tracąc precyzję z każdym kolejnym krokiem gry. Oznacza to, że taktyka nie jest zapisana wprost za pomocą prostych, płaskich cech wektorowych. Z kolei **sonda nieliniowa (MLP) z powodzeniem odzyskuje tę informację**, utrzymując bardzo wysoką skuteczność przez większość czasu trwania gry (podobnie jak w przypadku fizyki).

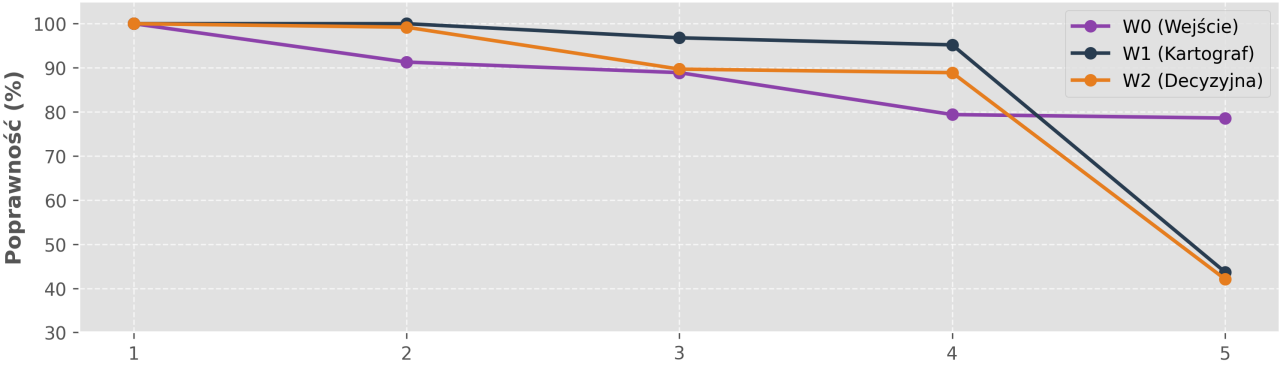
Prowadzi to do ważnego wniosku o budowie modelu językowego: świadomość "czyj to pionek" jest pojęciem głęboko spletanym nieliniowo wewnątrz 128-wymiarowej przestrzeni ukrytej. Sieć przechowuje tę relację w sposób rozproszony, a jej odczytanie wymaga zastosowania dodatkowych warstw transformacji (jak w MLP).



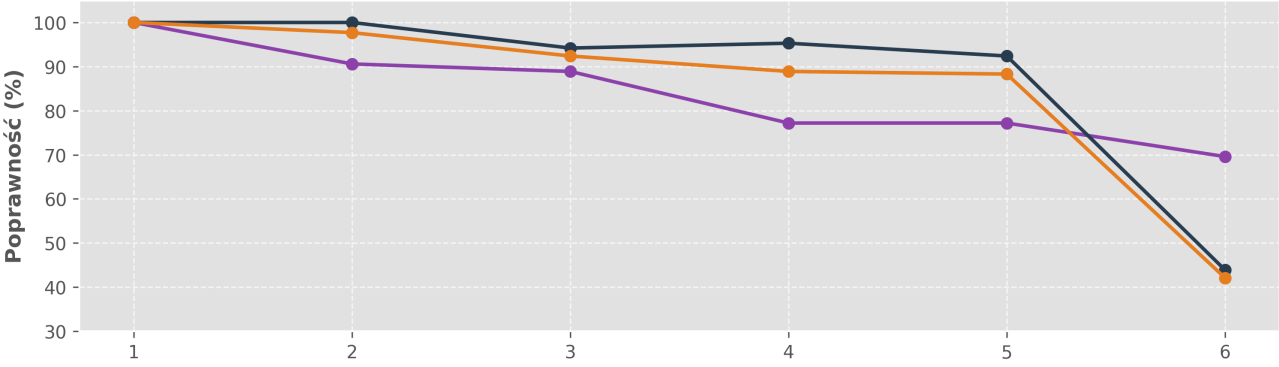
Wykres 3: Porównanie skuteczności sondy MLP dla zjawisk fizyki (wyższa skuteczność) oraz taktyki (nieco niższa skuteczność, świadcząca o wyższym stopniu skomplikowania problemu).

Taktyka - SONDA LINIOWA

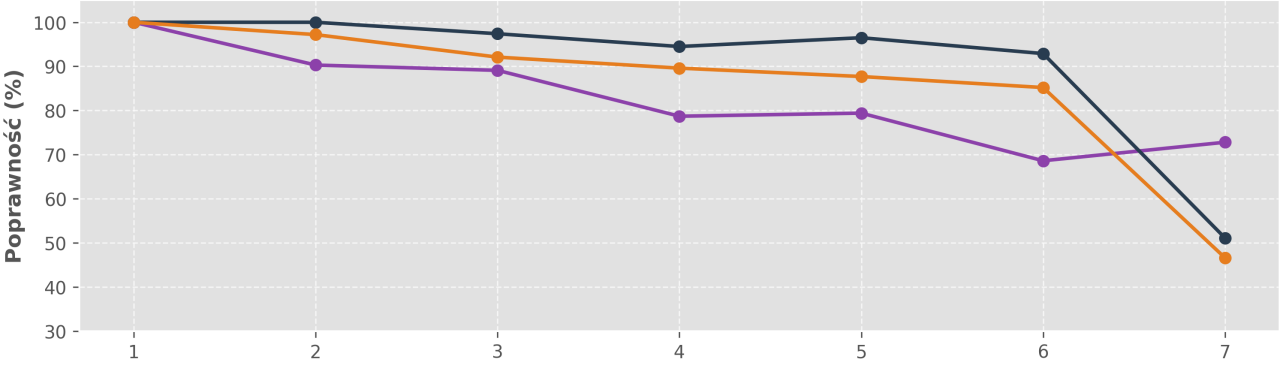
Gry kończące się w 5. ruchu



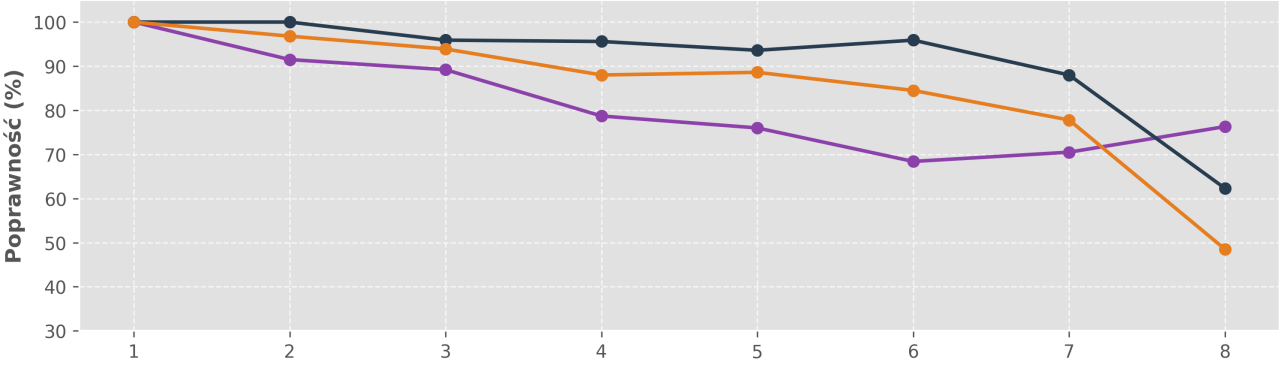
Gry kończące się w 6. ruchu



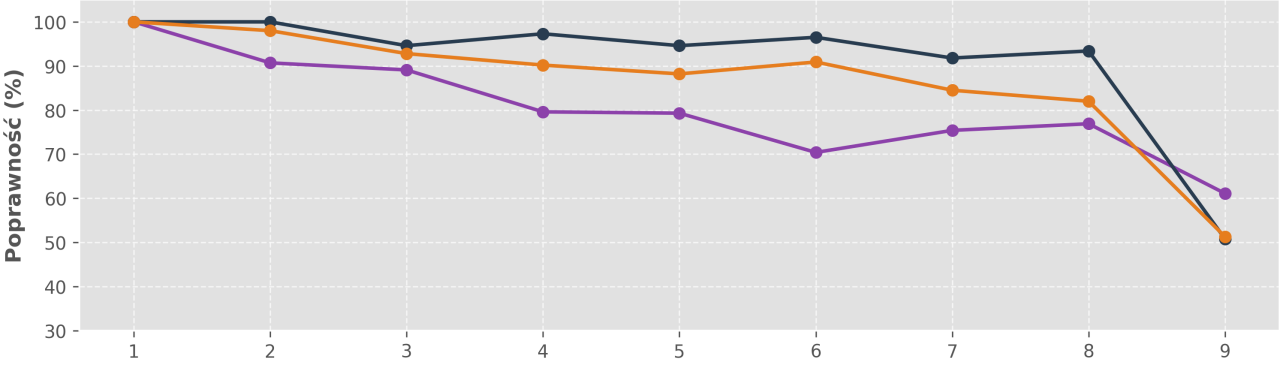
Gry kończące się w 7. ruchu



Gry kończące się w 8. ruchu



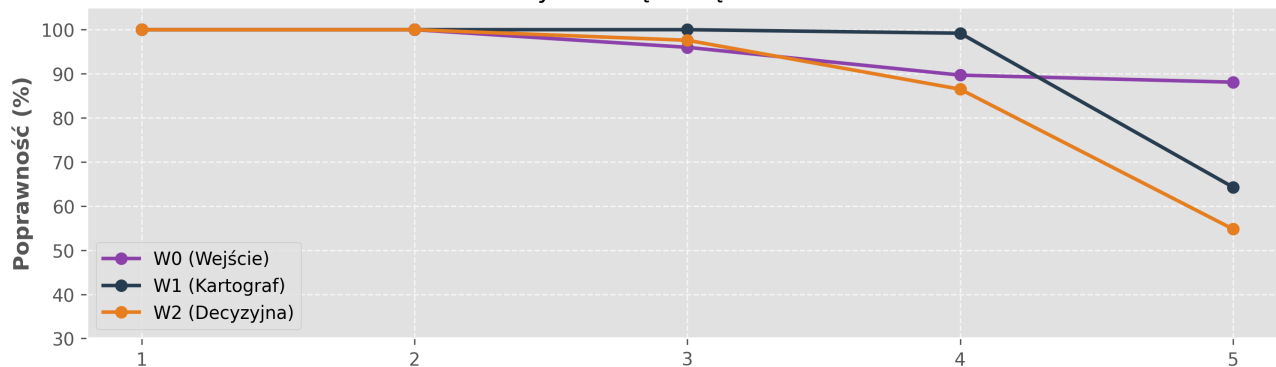
Gry kończące się w 9. ruchu



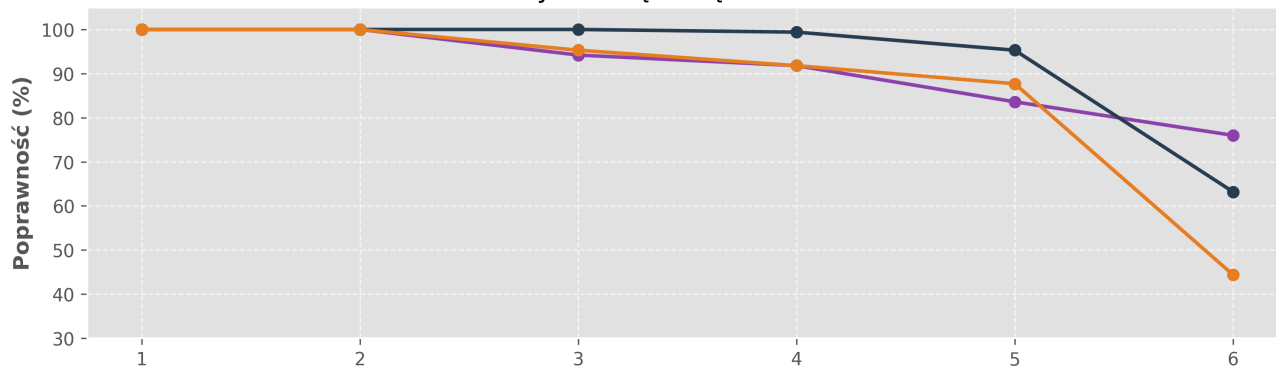
Wykres 4: Skuteczność sondy liniowej w zadaniu taktycznym. Widoczny jest problem z liniową separacją tej cechy, wykresy szybko opadają w miarę postępu gry.

Taktyka - SONDA MLP

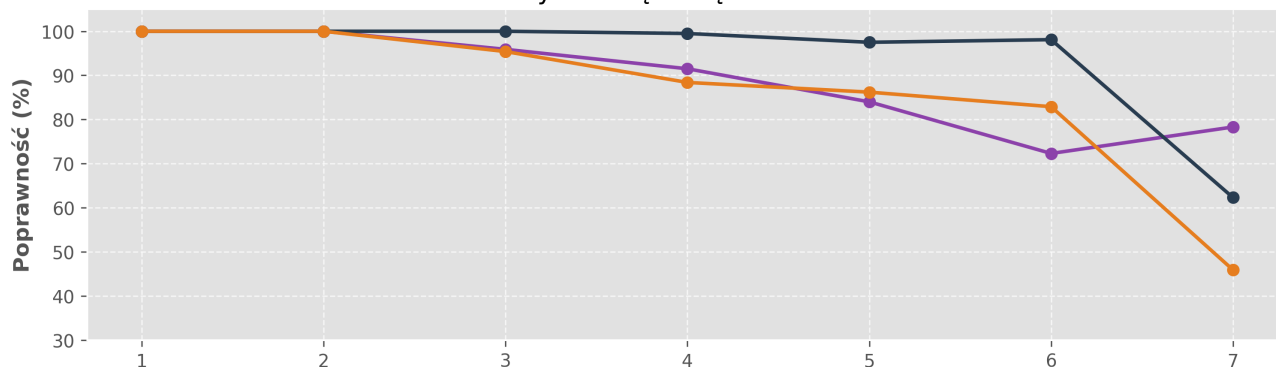
Gry kończące się w 5. ruchu



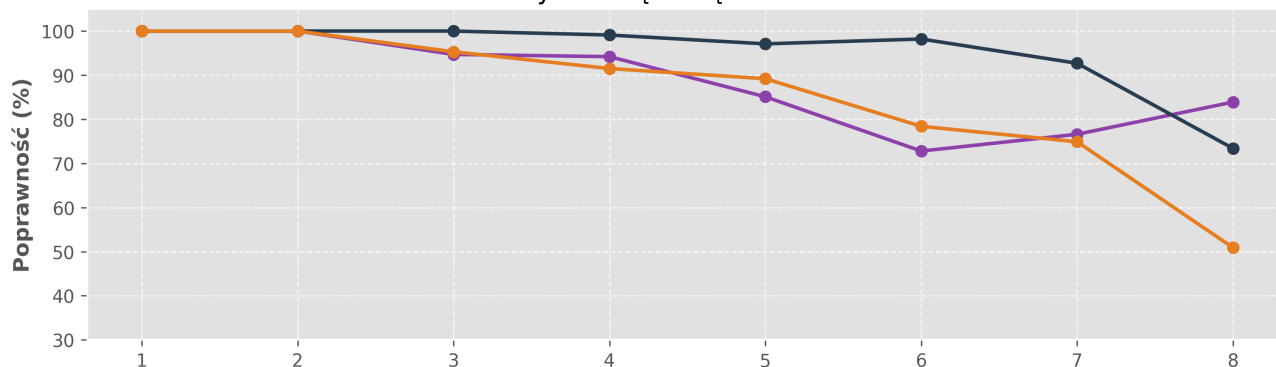
Gry kończące się w 6. ruchu



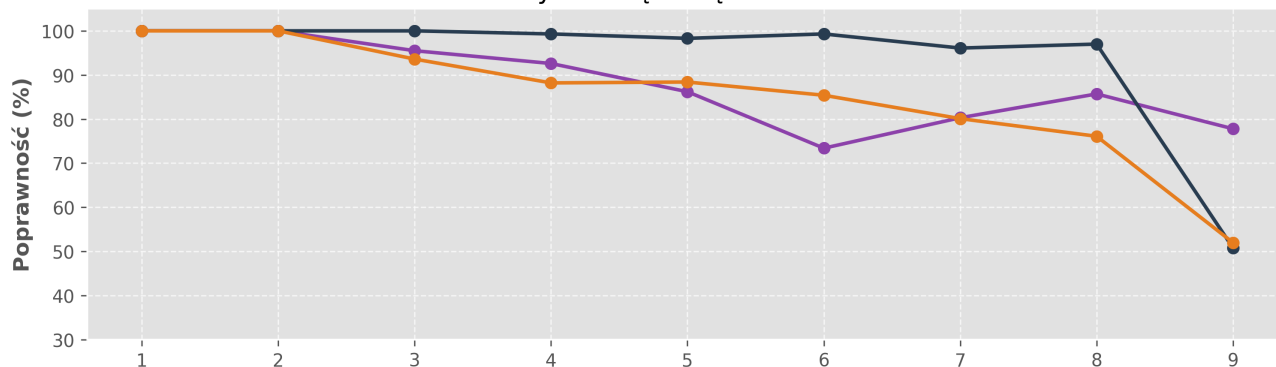
Gry kończące się w 7. ruchu



Gry kończące się w 8. ruchu



Gry kończące się w 9. ruchu



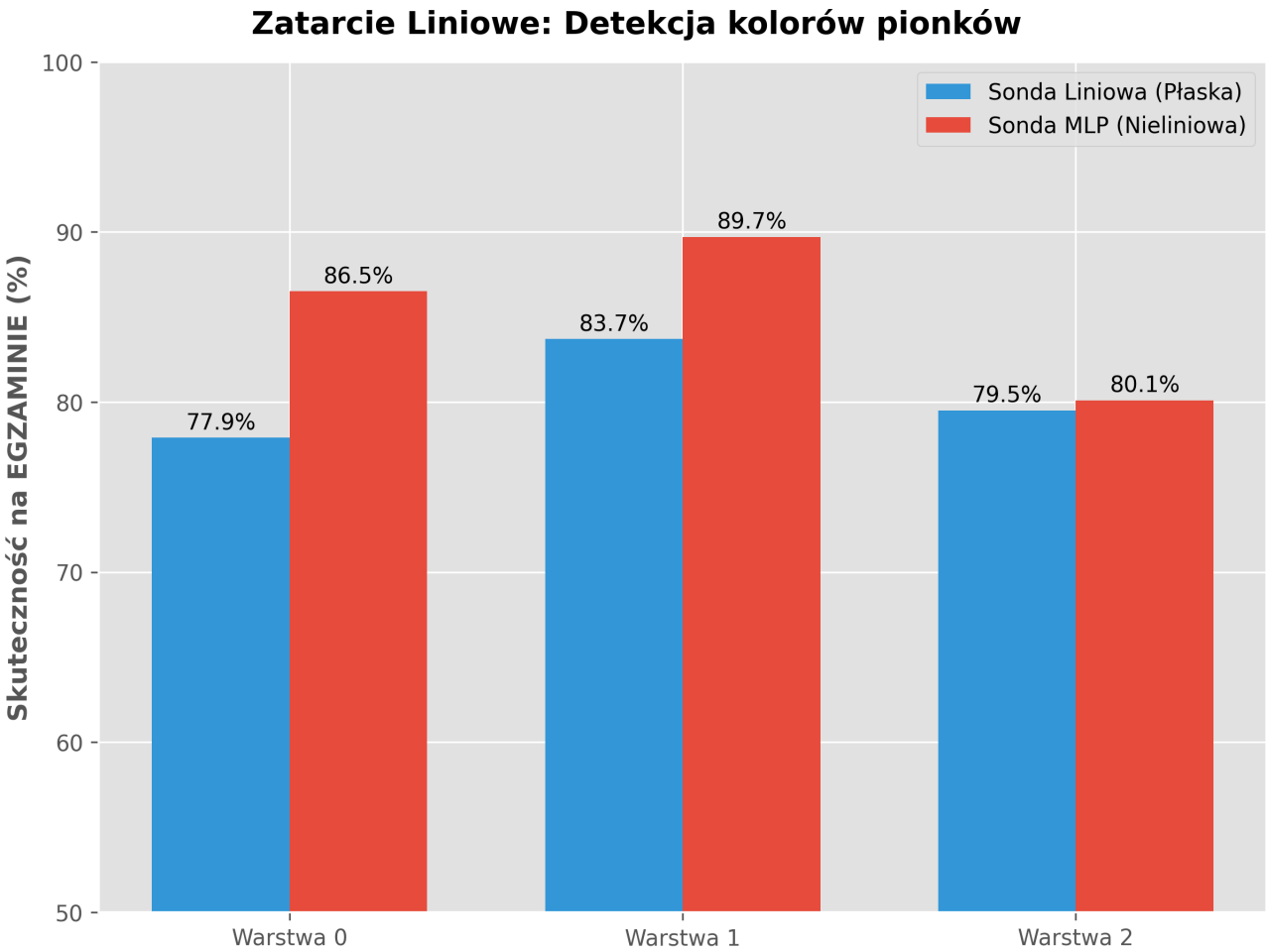
Wykres 5: Skuteczność sondy MLP w zadaniu taktycznym. Sieć potrafi wydobyć wiedzę o własności pionka, co potwierdza, że informacja ta istnieje w modelu, lecz ma charakter nieliniowy.

4.4. Zjawisko Zatarcia Liniowego (Attention Shift w Czasie)

Analiza aktywacji w czasie ujawniła zjawisko dynamicznego zarządzania pamięcią, które nazwano **Zatarciem Liniowym** (ang. *Linear Erasure*).

Zgrupowanie wyników względem długości gry pokazuje wyraźny wzorec: w szybkich rozgrywkach (5-6 ruchów) model utrzymuje perfekcyjnie ostrą mapę fizyki i taktyki. Jednak w grach długich (8-9 ruchów) precyzja odtwarzania starszych, zajętych pól gwałtownie spada w ostatnich fazach gry.

Oznacza to, że mechanizm *Self-Attention* optymalizuje zasoby na etapie *Late Game*. Sieć celowo "zapomina" nieistotne, stare ruchy, skupiając całą pojemność wektorową wyłącznie na "gorących polach" – niezbędnych do natychmiastowej wygranej lub zablokowania przeciwnika. Zjawisko to dowodzi, że sieć działa jak dynamiczny procesor uwagi, a nie statyczna baza danych.



Wykres 6: Skuteczność detekcji taktyki na egzaminie. Zatarcie liniowe drastycznie obniża skuteczność płaskich reprezentacji (niebieskie słupki). Głębokie sieci MLP (czerwone słupki) są odporniejsze i wciąż dekodują splątane dane z warstw ukrytych.

5. Interwencje Przyczynowe (Causal Interventions)

5.1. Definicja Dowodu Przyczynowo-Skutkowego

Sondowanie opisane w poprzednim rozdziale pozwala wykazać istnienie informacji w aktywacjach modelu, jednak samo w sobie nie dowodzi, że informacja ta jest wykorzystywana podczas podejmowania decyzji. W tym celu zastosowano metodę Interwencji Przyczynowych (*Causal Interventions*), polegającą na bezpośredniej modyfikacji aktywacji wewnętrznych podczas działania modelu.

Formalnie interwencję można opisać jako:

$$h' = h + \alpha v$$

gdzie:

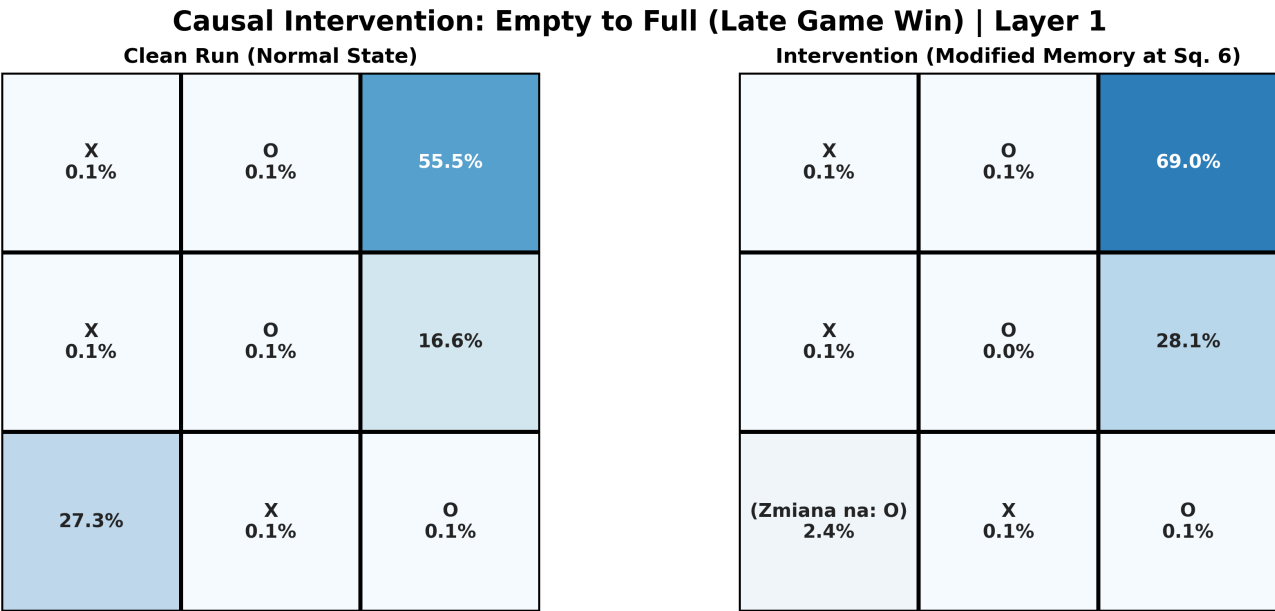
- h oznacza oryginalną aktywację,
- v oznacza kierunek konceptualny odpowiadający badanemu pojęciu,
- α jest współczynnikiem siły interwencji.

Jeżeli zmiana aktywacji prowadzi do przewidywalnej zmiany decyzji modelu, stanowi to eksperymentalną przesłankę przyczynową wskazującą, że dana reprezentacja bierze udział w procesie decyzyjnym.

5.2. Wymuszanie Decyzji

W pierwszym eksperymencie przeprowadzono próbę sztucznego oznaczenia pustego pola jako zajętego. W tym celu wykorzystano wektor konceptualny odpowiadający pojęciu „pole zajęte”.

Po wykonaniu interwencji zaobserwowano systematyczny spadek prawdopodobieństwa wyboru manipulowanego pola.



Wykres 7: Wpływ interwencji przyczynowej na dystrybucję prawdopodobieństwa (Warstwa 1). Sztuczna modyfikacja aktywacji wewnętrznych, wmawiająca modelowi, że lewe dolne pole jest zajęte, drastycznie obniża prawdopodobieństwo jego wyboru (spadek z 27,3% do 2,4%). Model dynamicznie przenosi masę prawdopodobieństwa na pozostałe puste pola (wzrost na prawym górnym i prawym środkowym polu), co stanowi bezpośredni dowód na przyczynowo-skutkową rolę badanej reprezentacji w procesie decyzyjnym sieci.

Uzyskane rezultaty wskazują, że reprezentacja legalności pól nie jest jedynie artefaktem możliwym do odczytania przez sondy, lecz aktywnie uczestniczy w generowaniu decyzji modelu.

5.3. Efekt Fali Uderzeniowej

Ponieważ aktywacje modelu operują w przestrzeni liczb zmiennoprzecinkowych, skuteczność interwencji zależy od jej amplitudy. W eksperymencie przeanalizowano wpływ różnych wartości współczynnika α na końcową dystrybucję prawdopodobieństwa.

Dla małych wartości α wpływ interwencji był ograniczony i ulegał tłumieniu przez kolejne transformacje sieci. Po przekroczeniu określonego progu zaobserwowano gwałtowną zmianę rozkładu wyjściowego, prowadzącą do trwałej zmiany preferowanego ruchu.

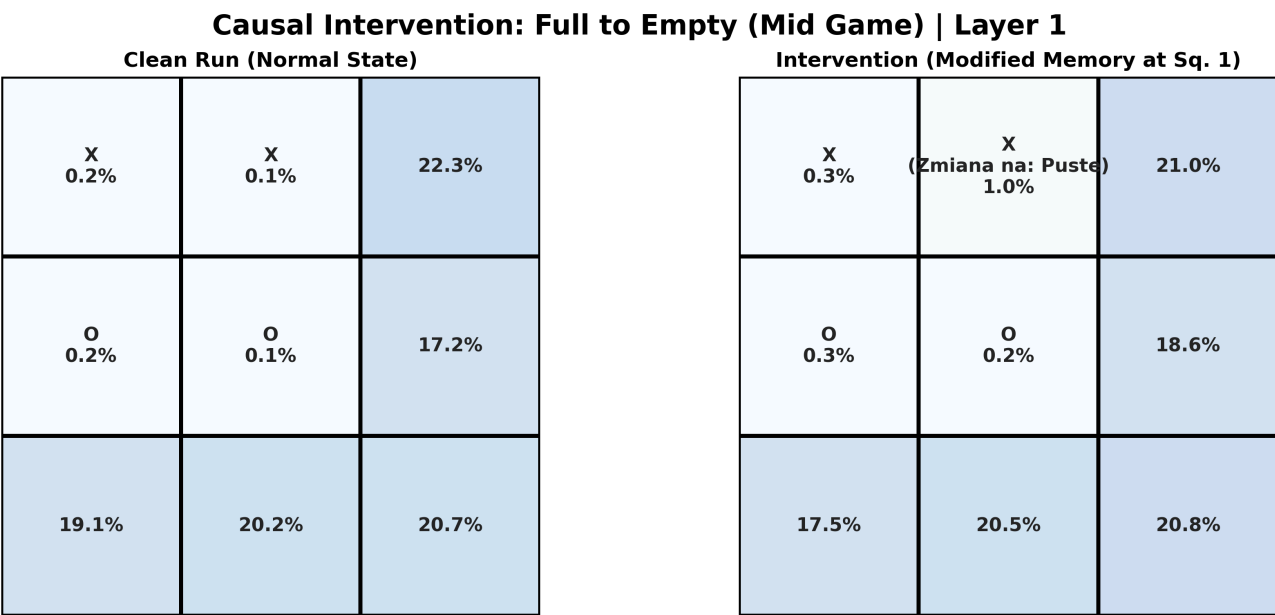
Analiza ta pozwoliła wyznaczyć empiryczne progi czułości modelu na manipulację jego reprezentacjami wewnętrznymi.

5.4. Asymetria Przyczynowa i Samonaprawa

Najbardziej interesujące wyniki uzyskano podczas eksperymentu odwrotnego. Zamiast oznaczać pole puste jako zajęte, podjęto próbę przekonania modelu, że pole zajęte jest wolne.

Zaobserwowano konsekwentny brak podatności na tę manipulację. Próba oszustwa poniosła fiasko na wszystkich etapach myślenia modelu:

- **Warstwa L0:** skuteczność interwencji = 0%
- **Warstwa L1:** skuteczność interwencji = 0.9%
- **Warstwa L2:** skuteczność interwencji = 1.4%



Wykres 8: Efekt samonaprawy (self-healing) i asymetria przyczynowa (Warstwa 1). Próba sztucznego przekonania modelu, że zajęte pole (górne środkowe) jest wolne, okazuje się w dużej mierze nieskuteczna na wszystkich warstwach. Mimo interwencji, prawdopodobieństwo wyboru zmanipulowanego pola wzrasta z 0,1% do zaledwie 1,0%, podczas gdy sieć konsekwentnie rozkłada główną masę prawdopodobieństwa na faktycznie puste pola (utrzymując udziały rzędu 17–21%). Wynik ten ilustruje odporność warstw modelu na lokalne zaburzenia i stanowi wizualizację mechanizmów korekcyjnych opartych na redundantnych reprezentacjach stanu gry.

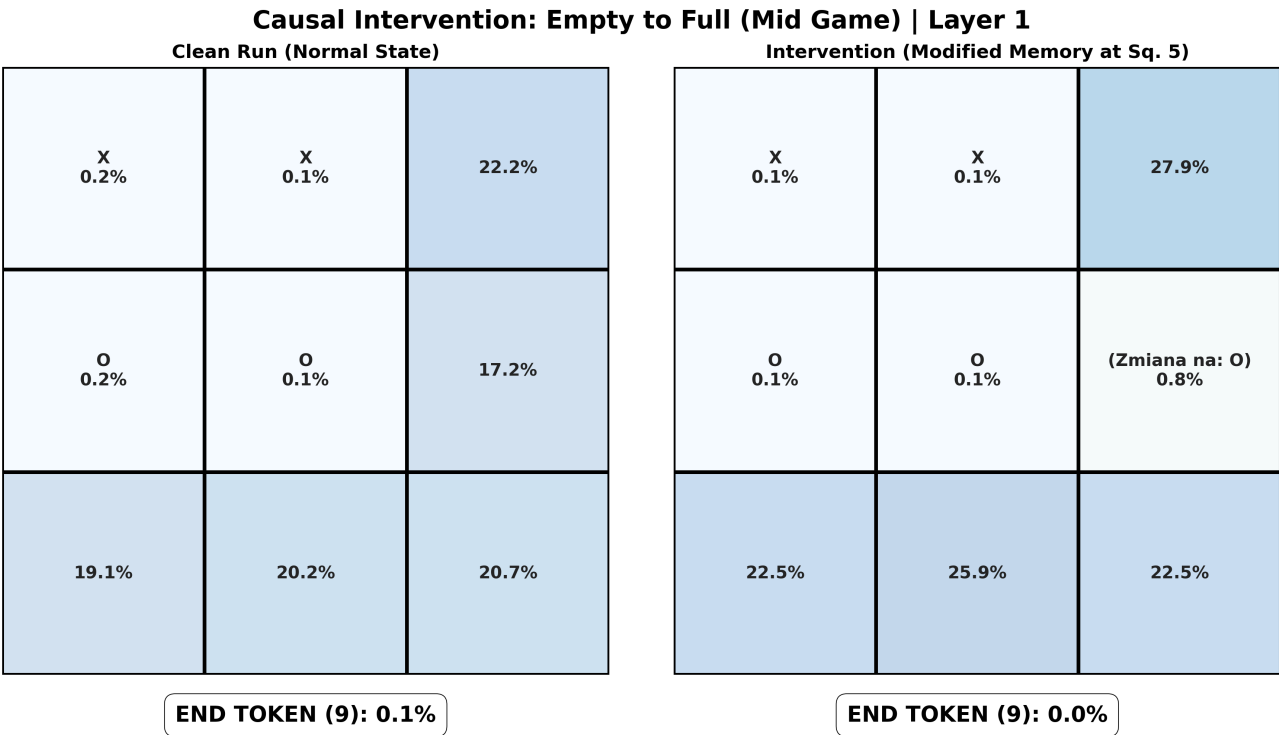
Wynik ten bezsprzecznie udowadnia istnienie mechanizmów samokorygujących. Zamiast polegać na jednym, podatnym na błędy miejscu w pamięci, architektura wykorzystuje **redundancję**. Zjawisko to można

wytłumaczyć działaniem obwodów uwagi (Self-Attention). Kiedy wstrzyknięta przez nas fałszywa mapa sugeruje, że pole jest wolne, mechanizm uwagi natychmiast weryfikuje to z twardą listą rozegranych wcześniej ruchów. Widząc konflikt danych, sieć samodzielnie naprawia błąd w pamięci i twardo odrzuca nielegalny ruch. Nazywamy to zjawiskiem **Self-Healing**.

5.5. Paradoks Stanu Niemożliwego i Modularność Sieci

W toku badań z interwencjami przeprowadzono dodatkowy eksperyment wprowadzający sieć w celowy błąd logiczny. W trakcie tury jednego z graczy, sztucznie zmodyfikowano wewnętrzną mapę przestrzenną modelu tak, aby na planszy pojawiła się gotowa, wygrywająca linia należąca do przeciwnika. W rzeczywistej rozgrywce taka sytuacja wymusiłaby natychmiastowe zakończenie partii (wygenerowanie tokenu 9).

Wyniki ukazały niespodziewane zachowanie sieci. Mimo istnienia linii wygrywającej w zmanipulowanej pamięci obrazu planszy, model nie podniósł prawdopodobieństwa wygenerowania tokenu końca gry.



Wykres 9: Paradoks Stanu Niemożliwego. Mimo sztucznego utworzenia wygrywającej linii dla gracza O (pola 3, 4, 5) na planszy interwencji, wskaźnik końca gry "END TOKEN (9)" pozostaje uśpiony. Model całkowicie ignoruje mapę przestrzenną w kontekście ewaluacji zwycięstwa.

Zjawisko to, nazwane "Paradoksem Stanu Niemożliwego", jest niezwykle silnym dowodem na **Modularność Reprezentacji**. Badania udowodniły, że "mózg" modelu zorganizowany jest w formie wyspecjalizowanych obwodów:

- Obwód Mapy Przestrzennej:** Kontroluje fizykę gry, służąc wyłącznie do identyfikacji wolnych i zajętych pól. Oszukanie tego obwodu skutecznie zablokowało możliwość wykonania ruchu na zmanipulowane pole.
- Obwód Sędziego:** Weryfikuje warunek zwycięstwa na zupełnie innej płaszczyźnie danych. Moduł ten całkowicie ignoruje trójwymiarową mapę planszy, poszukując wzorców wygranej wyłącznie poprzez skanowanie liniowej historii znaków (tekstu na wejściu). Ponieważ w oryginalnej historii ruchów linia wygrywająca nie istniała, sędzia nie zezwolił na zakończenie partii.

Udowadnia to fundamentalną zasadę działania małych sieci Transformer – model nie rozważa gry w sposób scentralizowany, lecz dzieli zadania na zbiór mniejszych, niezależnych od siebie programów wykonawczych.

6. Podsumowanie i Wnioski

6.1. Weryfikacja Tezy Głównej

Projekt jednoznacznie potwierdza postawioną tezę badawczą. Autoregresyjny model językowy, trenowany bez jakiegokolwiek strategii wygrywającej i wyłącznie na losowych sekwencjach poprawnych ruchów, nie zapamiętuje "w ciemno" statystycznych wzorców tekstu. Zamiast tego, w procesie optymalizacji i unikania błędów (kar za wygenerowanie ruchu w niedozwolone miejsce), model samoistnie wykształca **Emergent World Representations** [Wyłaniające się Reprezentacje Świata - wewnętrzny, zgeometryzowany model fizycznej rzeczywistości].

Zdekodowanie tej mapy za pomocą sond liniowych oraz weryfikacja jej działania za pomocą interwencji przyczynowych stanowi twardy dowód. Pokazuje on, że sieć przed wygenerowaniem kolejnego znaku aktywnie "rozumie" fizyczny układ planszy i opiera na nim swoje decyzje.

6.2. Odporność Architektury Transformer (Redundancja i Obwody Zapasowe)

Badania nad modyfikacją pamięci sieci ujawniły nieoczekiwaną dojrzałość i złożoność w tak małej architekturze. Próby sztucznego "pozwolenia" modelowi na zrobienie ruchu w zajęte już pole doprowadziły do odkrycia zjawiska **Self-Healing** [Samonaprawa Sieci - zdolność modelu do korygowania fałszywych informacji wewnątrz własnej pamięci].

Udowodniono, że wszystkie warstwy sieci aktywnie współpracują z mechanizmem **Self-Attention** [Głowy Uwagi - moduły skanujące początkową historię ruchów podaną na wejściu]. Kiedy nasza zmanipulowana mapa przestrzenna sugerowała, że zajęte pole jest puste, głowy uwagi natychmiast weryfikowały to z twardą listą wcześniejszych ruchów i skutecznie blokowały ten błąd. Nasze oszustwo zostało odrzucone na każdym etapie przetwarzania (L0, L1, L2). To jest twardy dowód na to, że architektura Transformer naturalnie wykształca **Redundancję** [Istnienie obwodów zapasowych, które działają niezależnie, aby chronić system przed awarią i błędnymi decyzjami].

6.3. Modularność Reprezentacji (Niezależne Obwody)

Kluczowym odkryciem badawczym było udowodnienie, że architektura dzieli proces decyzyjny na izolowane moduły. Udana oszukanie wewnętrznej reprezentacji przestrzennej ("radaru pustych pól") pozostało całkowicie zignorowane przez obwód odpowiadający za ocenę zakończenia gry. Oznacza to, że różne zadania optymalizacyjne w środowisku wykształciły w wagach sieci oddzielne, specjalistyczne ścieżki wnioskowania. Taka modułowa budowa to kolejny filar tłumaczący stabilność i bezpieczeństwo sieci w środowisku ustrukturyzowanym.

6.4. Znaczenie Projektu dla Dziedziny Interpretowalności AI

Wyniki tego projektu stanowią istotny, praktyczny wkład w rozwój dziedziny **Mechanistic Interpretability** [Mechanistyczna Interpretowalność - nauka o inżynierii odwrotnej i dekodowaniu czarnych skrzynek sztucznej inteligencji].

Praca ta dostarcza empirycznych dowodów na to, że nawet proste modele językowe realizujące zadanie przewidywania kolejnego znaku (Next-Token Prediction) są zdolne do budowania precyzyjnych i odpornych na manipulacje symulacji swojego środowiska. Zrozumienie, w jaki sposób modele te uczą się pojęć przestrzennych i jak ich wewnętrzne obwody same korygują błędy, jest kluczowym krokiem w stronę tworzenia bezpiecznej i przewidywalnej sztucznej inteligencji. Metodologia użyta w tej pracy, polegająca na łączeniu sond wektorowych z interwencjami przyczynowymi, z powodzeniem może zostać przeskalowana do analizy znacznie potężniejszych sieci neuronowych.